

Experiment 1

Aim:

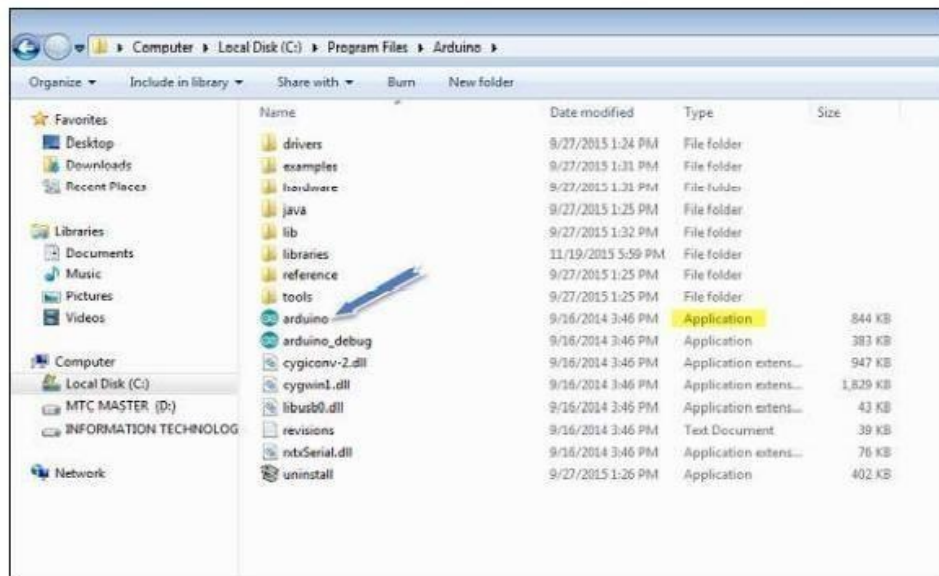
To familiarize myself with Arduino and perform the necessary software installation.

Description:

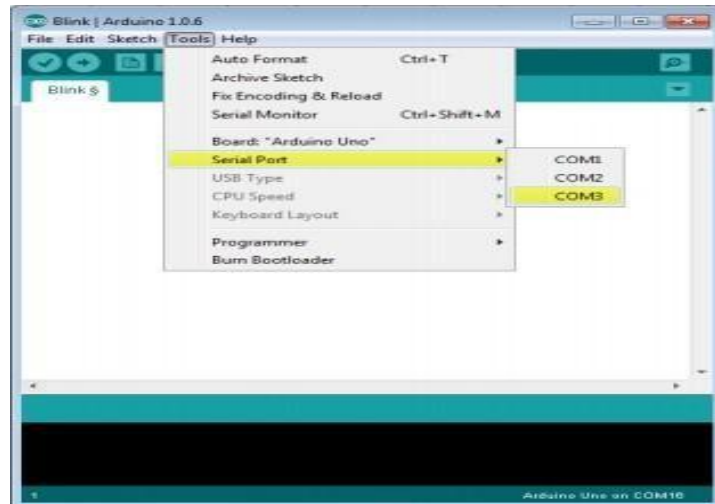
In this experiment, I learned about Arduino and Raspberry Pi platforms. I installed the Arduino IDE software, explored its environment, and executed a sample program. This helped me understand how to set up the development environment and upload a basic program to the Arduino board.

Procedure:

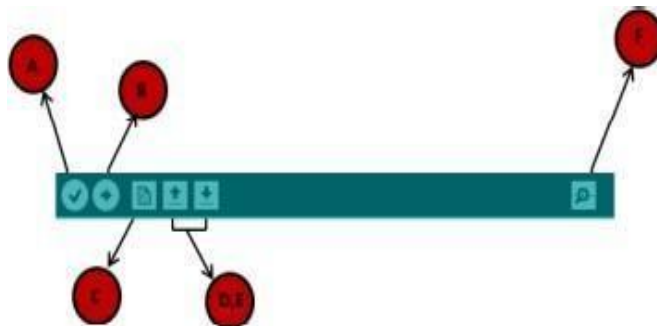
1. I downloaded the Arduino IDE software from the official website "<https://www.arduino.cc>".
2. I launched the Arduino IDE on my system.
3. After the download, I unzipped the folder and located the application file with the infinity label (application.exe). I double-clicked on the icon to start the IDE.



4. Once the IDE opened, I had two options: create a new project or open an existing example. I chose to open an existing project by navigating to **File** → **Examples** → **Basics** → **Blink**. This program makes the LED turn on and off with a time delay.

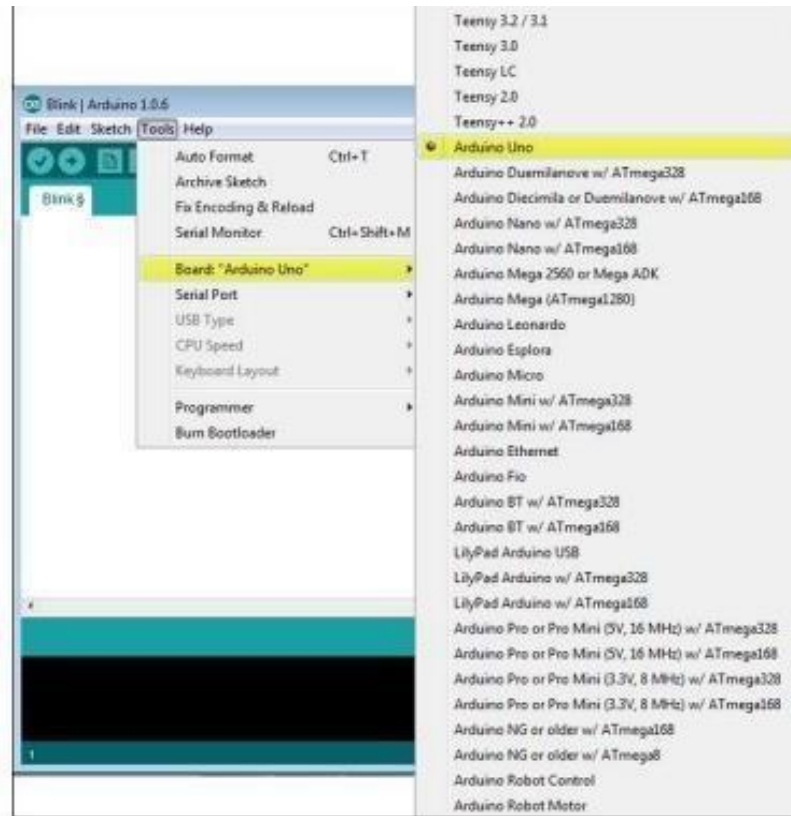


5. I selected my Arduino board by going to **Tools** → **Board** and choosing **Arduino Uno**, as it matched the board I was using.
6. I selected the correct serial port by navigating to **Tools** → **Serial Port**. I verified this by disconnecting and reconnecting my Arduino board and then choosing the correct COM port.
7. I became familiar with the Arduino IDE toolbar functions:



- A. Verify (check for errors)
- B. Upload (send program to the board)
- C. New Sketch (create a new project)
- D. Open (open existing examples)
- E. Save (save the sketch)
- F. Serial Monitor (communicate with the board)

8. clicked the **Upload** button to send the Blink program to the Arduino board.



9. observed the RX and TX LEDs flashing on the board. After a few seconds, the message **“Done uploading”** appeared in the status bar.

Result:

I successfully installed the Arduino IDE, configured the board and serial port, and uploaded the Blink program. The Arduino Uno executed the program correctly, and the onboard LED blinked with a time delay.

Conclusion:

Through this experiment, I understood how to install and set up the Arduino IDE, select the appropriate board and port, and upload a program to the Arduino. This gave me the basic knowledge required to start working on IoT applications.

Experiment 2

Aim:

To interface an LED/Buzzer with Arduino and write a program to turn ON the LED for 1 second after every 2 seconds.

Requirements:

- Standard 5mm LED
- One Resistor
- Arduino Uno microcontroller board (ATmega328p)
- Jumper wires

Description:

In this experiment, I gained practical experience in interfacing an LED with the Arduino Uno microcontroller. The LED was connected through a resistor to protect it from excessive current, and a program was developed to make the LED blink continuously, remaining ON for 1 second and OFF for 2 seconds. This experiment helped me understand the operation of digital pins and how they can be used to control external components in IoT applications.

Procedure:

1. I opened www.wokwi.com in my browser and selected the **Arduino Uno** microcontroller.
2. In the simulation section, I added the required components (LED, resistor, jumper wires, and Arduino Uno) by clicking the “+” symbol.
3. I used jumper wires to make the connections: I connected the **Anode (A-pin)** of the LED to **digital pin 3** of the Arduino board through the resistor, and I connected the **Cathode (C-pin)** of the LED to **Ground (GND)**.
4. I wrote the following program in the **sketch.ino** file:

Code:

```
/*  
  Experiment: Interfacing an LED with Arduino Uno  
  Purpose: To turn ON the LED for 1 second after every 2 seconds  
  Name: S NANDAKISHORE  
  Roll No: 25M11MC241  
*/  
  
void setup() {  
    pinMode(3, OUTPUT);    // Set pin 3 as OUTPUT for LED  
}
```

```

void loop() {
    digitalWrite(3, HIGH);    // Turn LED ON
    delay(1000);              // Keep ON for 1 second
    digitalWrite(3, LOW);     // Turn LED OFF
    delay(2000);              // Keep OFF for 2 seconds
}

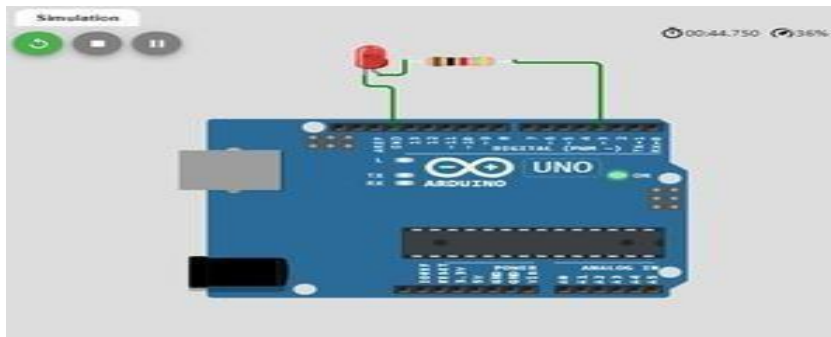
```

5. I compiled and uploaded the program to the Arduino Uno board in the simulation.
6. I observed the LED blinking – it turned ON for 1 second and remained OFF for 2 seconds repeatedly.

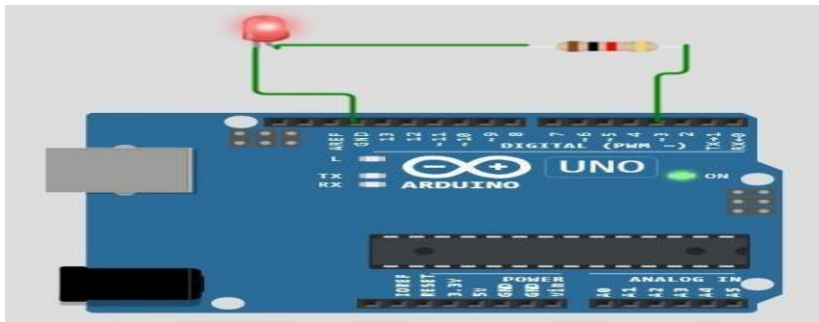
Result:

I successfully interfaced the LED with the Arduino Uno board and observed that the LED turned ON for 1 second and remained OFF for 2 seconds, working as expected.

Before:



After:



Conclusion:

Through this experiment, I learned how to interface external output devices like LEDs with Arduino and control them using digital pins. I also understood how to use `pinMode()`, `digitalWrite()`, and `delay()` functions for timing control in Arduino programs.

Experiment 3

Aim:

To interface a Push Button/Digital Sensor (IR/LDR) with Arduino and write a program to turn ON the LED when the push button is pressed or at sensor detection.

Requirements:

- Standard 5mm LED
- Two Resistors
- Arduino Uno microcontroller board (ATmega328p)
- Jumper wires
- Push button

Description:

In this experiment, I interfaced a push button (digital input device) with the Arduino Uno microcontroller to control an LED. The LED turned ON when the push button was pressed. This experiment helped me understand how to use digital inputs in Arduino, read button states, and control output devices based on user interaction or sensor detection.

Procedure:

1. I opened www.wokwi.com in my browser and selected the **Arduino Uno** microcontroller.
2. In the simulation section, I added the required components (LED, resistors, jumper wires, push button, and Arduino Uno) by clicking the “+” symbol.
3. I connected the **Anode (A-pin)** of the LED to **digital pin 3** of the Arduino microcontroller through **resistor1**, and I connected the **Cathode (C-pin)** of the LED to **Ground (GND)**.
4. I provided a **5V power supply** to the push button pin **1:1L**.
5. I connected the push button pin **1:1R** both to **Ground through resistor2** and to **digital pin 5** of the Arduino board.
6. I wrote the following program in **sketch.ino**:

Code:

/*Experiment 3: Interfacing a Push Button/Digital Sensor with Arduino Uno
Purpose: To turn ON the LED when the push button is pressed or at sensor
detection

Name: S NANDAKISHORE

Roll No: 25M11MC241

*/

```
int flag = 0;

void setup() {
  pinMode(3, OUTPUT);    // LED connected to pin 3
  pinMode(5, INPUT);     // Push button connected to pin 5
}

void loop() {
  int value = digitalRead(5);    // Read push button state

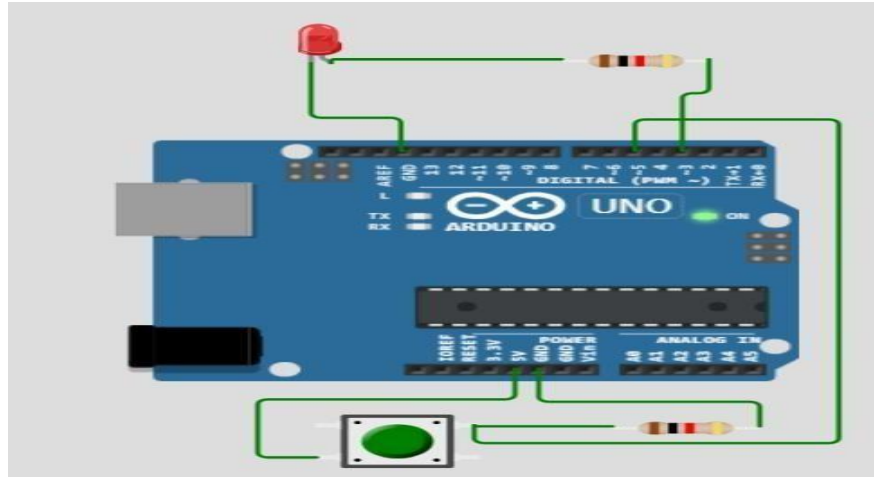
  if (value == 1 && flag == 0) {
    digitalWrite(3, HIGH);    // Turn LED ON
    delay(1000);
    flag = 1;
  }
  else if (value == 1 && flag == 1) {
    digitalWrite(3, LOW);     // Turn LED OFF
    delay(1000);
    flag = 0;
  }
}
```

7. I compiled and uploaded the program to the Arduino Uno board in the simulation.
8. I observed that when I pressed the push button, the LED turned ON. On pressing it again, the LED turned OFF.

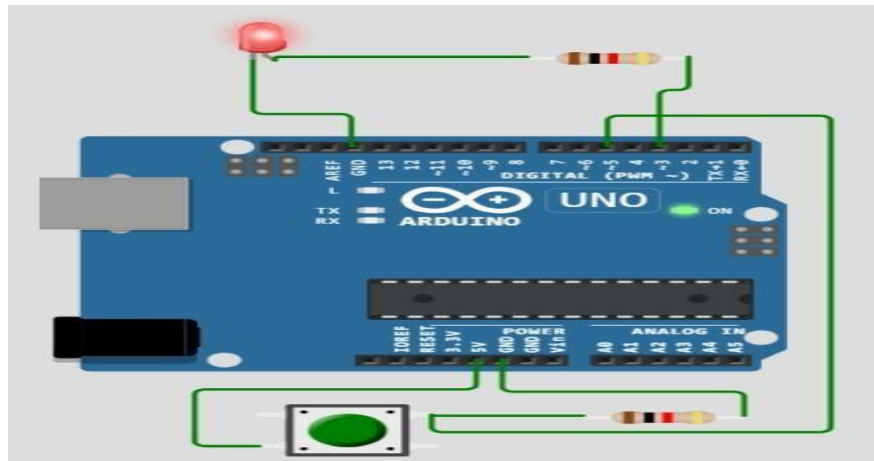
Result:

I successfully interfaced a push button with the Arduino Uno. When I pressed the push button, the LED turned ON, and on pressing again, it turned OFF.

Before:



After:



Conclusion:

Through this experiment, I learned how to interface a push button (digital input) with Arduino and use its state to control an output device. I understood the use of `digitalRead()` for inputs and how conditional logic can toggle device states in IoT applications.

Experiment 4

Aim:

To interface the DHT22 sensor with Arduino and write a program to print temperature and humidity readings.

Requirements:

- DHT22 sensor
- Arduino Uno microcontroller board (ATmega328p)
- Jumper wires

Description:

In this experiment, I interfaced a DHT22 sensor with the Arduino Uno to measure environmental temperature and humidity. The DHT22 sensor is widely used in IoT projects for weather monitoring and home automation.

Procedure:

1. I opened www.wokwi.com in my browser and selected the **Arduino Uno** microcontroller.
2. In the simulation section, I added the required components (DHT22 sensor, jumper wires, and Arduino Uno) by clicking the “+” symbol.
3. I made the following connections using jumper wires:
 - I connected **VCC pin of DHT22** to **5V power supply** of Arduino.
 - I connected **SDA pin of DHT22** to **digital pin 3** of Arduino Uno.
 - I connected **GND pin of DHT22** to **Ground (GND)** of Arduino Uno.
4. I wrote the following program in **sketch.ino**:

Code:

```
/*  
Experiment 4: Interfacing the DHT22 sensor with Arduino Uno Purpose:  
To print temperature and humidity readings  
Name:S NANDAKISHORE Roll  
No: 25M11MC241  
*/  
  
#include <Adafruit_Sensor.h>  
#include <DHT.h>  
#include <DHT_U.h>
```

```

DHT Sensor(3, DHT22);

void setup() {
  Serial.begin(9600);
  Sensor.begin();

  // Display student details before starting readings

  Serial.println("=====");
  Serial.println(" Experiment 4: DHT22 Sensor Interface ");
  Serial.println(" Name      : S NANDAKISHORE");
  Serial.println(" Roll No : 25M11MC241");

  Serial.println("=====");
  delay(2000); // Pause for 2 seconds before readings
}

void loop() {
  float tempr = Sensor.readTemperature();
  float humid = Sensor.readHumidity();

  Serial.print("Temperature = ");
  Serial.print(tempr);
  Serial.print(" °C, ");
  Serial.print("Humidity = ");
  Serial.print(humid);
  Serial.println(" %");

  delay(1000); // Update every second
}

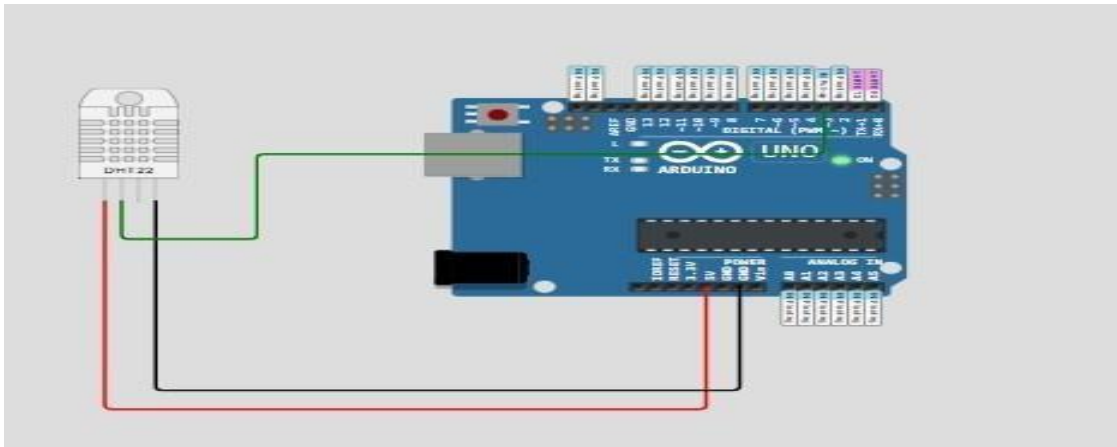
```

5. I compiled and uploaded the program to the Arduino Uno in the simulation.
6. I opened the **Serial Monitor** and observed the temperature and humidity values displayed continuously.

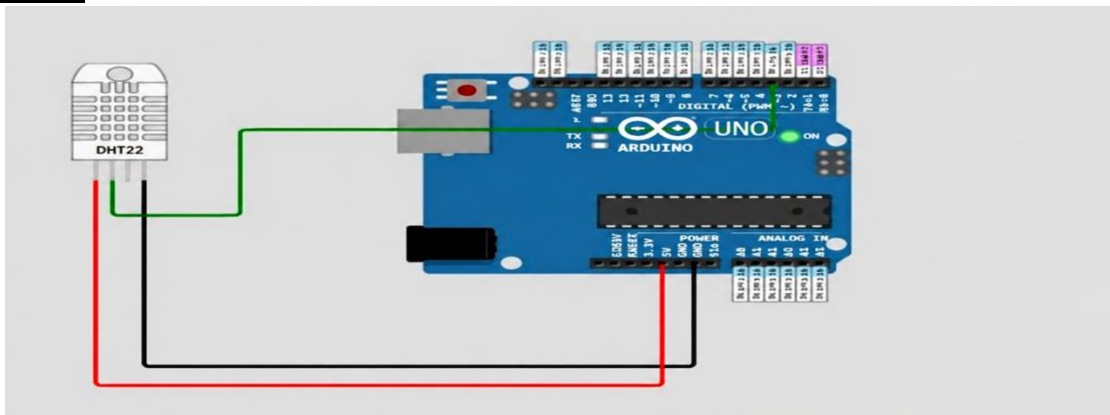
Result:

I successfully interfaced the DHT22 sensor with the Arduino Uno and observed real-time **temperature and humidity readings** displayed on the serial monitor.

Before:



After:



Experiment 4: DHT22 Sensor Interface

Name : S NANDAKISHORE

Roll No : 25M11MC241

```
=====
Temperature = 24.00 °C, Humidity = 40.00 %
Temperature = 24.00 °C, Humidity = 40.00 %
```

Conclusion:

From this experiment, I understood how to interface digital sensors like the DHT22 with Arduino. I learned to use sensor libraries, read environmental parameters, and display them through the serial monitor, which is useful in IoT-based weather monitoring and automation systems.

Experiment 5

Aim:

To interface an OLED with Arduino and write a program to print temperature and humidity readings on it.

Requirements:

- DHT22 sensor
- Arduino Uno microcontroller board (ATmega328p)
- Jumper wires
- SSD1306 OLED display

Description:

In this experiment, I interfaced an OLED display with the Arduino Uno along with the DHT22 sensor. The DHT22 provided real-time temperature and humidity values, and the OLED displayed these readings. This experiment helped me understand how to use I²C communication in Arduino and how to display sensor data on graphical displays, which is useful in IoT applications like smart weather stations and home automation systems.

Procedure:

1. I opened www.wokwi.com in my browser and selected the **Arduino Uno** microcontroller.
2. In the simulation section, I added the required components (DHT22 sensor, SSD1306 OLED, jumper wires, and Arduino Uno) by clicking the “+” symbol.
3. I made the following connections using jumper wires:
 - a) I connected **VCC pin of DHT22** to **5V power supply** of Arduino.
 - b) I connected **SDA pin of DHT22** to **digital pin 3** of Arduino.
 - c) I connected **GND pin of DHT22** to **Ground (GND)** of Arduino.
 - d) I connected the **OLED display** as follows:
 - I. OLED DATA → Analog pin **A4**
 - II. OLED CLK → Analog pin **A5**
 - III. OLED VIN → **5V power**
 - IV. OLED GND → **Ground**
4. I downloaded and included the **Adafruit SSD1306**, **Adafruit GFX**, and **DHT sensor libraries** in the Arduino IDE.
5. I wrote the following program in **sketch.ino**:

```

/*
  Experiment 5: Interfacing OLED with Arduino Uno
  Purpose: To print temperature and humidity readings on OLED
  Name: S NANDAKISHORE
  Roll No: 25M11MC241
*/

#include <Adafruit_Sensor.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>
#include <DHT_U.h>
#include <Wire.h>

// OLED settings
Adafruit_SSD1306 display(128, 64, &Wire);

// DHT22 settings
DHT Sensor(3, DHT22);

void setup() {
  Serial.begin(9600);
  Sensor.begin();

  // Initialize OLED
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("SSD1306 allocation failed");
    for (;;)
  }
  display.clearDisplay();
  display.setTextColor(WHITE);

  // Show student details first
  display.setTextSize(1);
  display.setCursor(0, 10);
  display.println("Experiment 5");
  display.println("OLED with DHT22");
  display.println("-----");
  display.println("Name: S NANDAKISHORE");
  display.println("Roll No: 25M11MC241");
  display.display();

  delay(3000); // Show info for 3 seconds
  display.clearDisplay();
}

```

```

void loop() {
    float tempr = Sensor.readTemperature();
    float humid = Sensor.readHumidity();

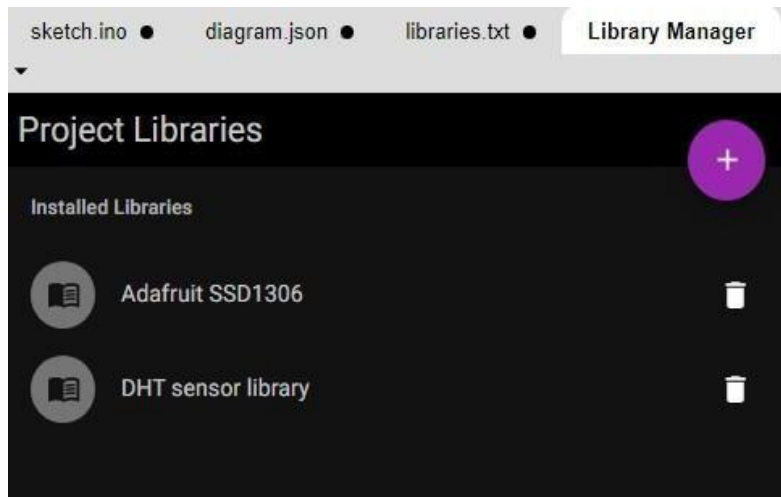
    // Display sensor readings
    display.setTextSize(1);
    display.setCursor(0, 10);
    display.print("Temp: ");
    display.print(tempr);
    display.println(" C");

    display.print("Humidity: ");
    display.print(humid);
    display.println(" %");

    display.display();
    delay(1000);    // Update every second
    display.clearDisplay();
}

```

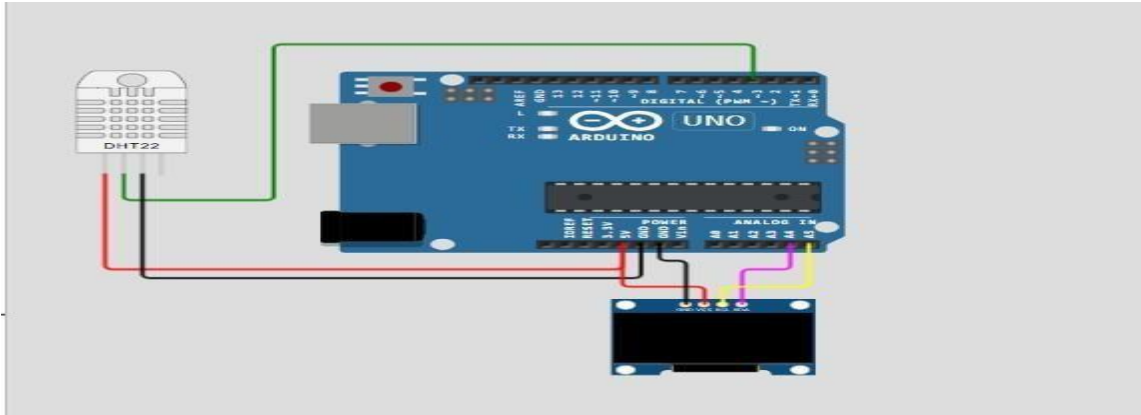
6. I compiled and uploaded the program to the Arduino Uno in the simulation.
7. I observed that the OLED displayed the real-time **temperature and humidity values** from the DHT22 sensor.



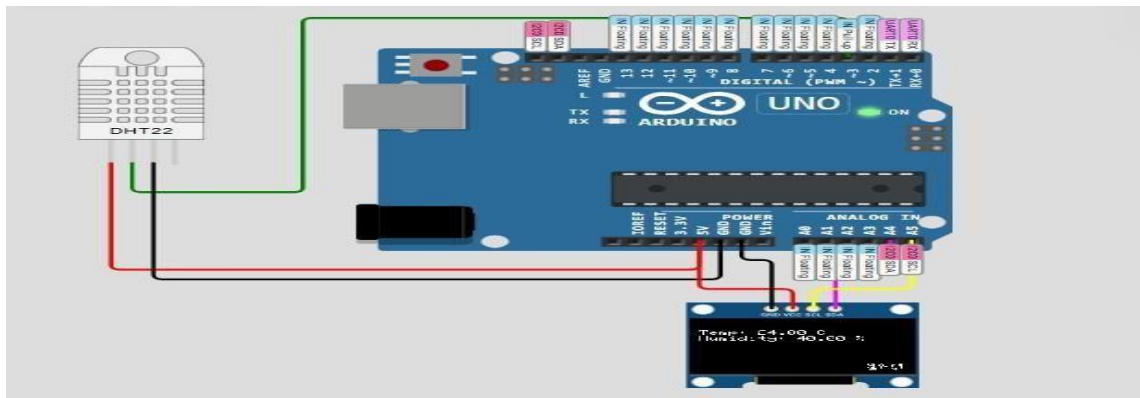
Result:

I successfully interfaced the OLED display with the Arduino Uno and observed that the real-time temperature and humidity readings from the DHT22 sensor were displayed on the OLED screen.

Before:



After:



Conclusion:

Through this experiment, I learned how to interface and program an OLED display with Arduino using the I²C protocol. I also understood how to collect real-time data from a sensor and display it on a graphical interface, which is an essential part of IoT applications.

Experiment 6

Aim:

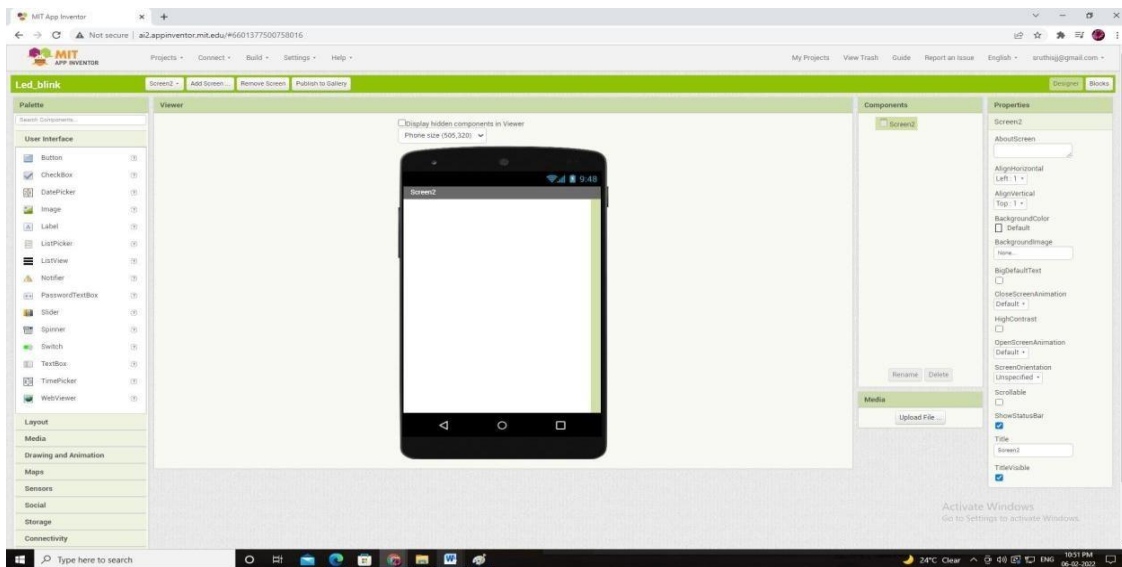
To interface Bluetooth with Arduino/Raspberry Pi and write a program to send sensor data to a smartphone using Bluetooth.

Description:

In this experiment, I interfaced a Bluetooth module with the Arduino microcontroller to enable wireless transmission of sensor data to a smartphone. Using MIT App Inventor, I developed an Android application that established a Bluetooth connection with the Arduino and displayed the received data. This experiment helped me understand wireless communication between IoT devices and mobile applications, along with the practical use of Bluetooth technology for remote data monitoring and device interaction.

Procedure:

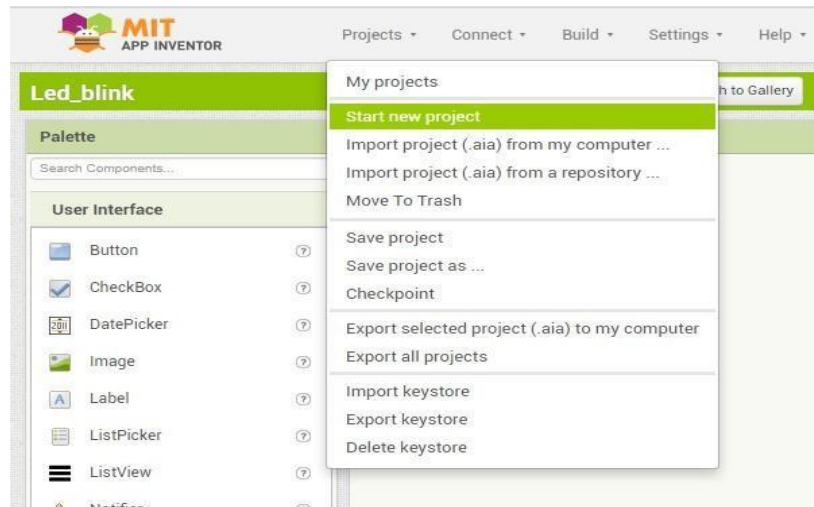
1. I opened **MIT App Inventor** in Google Chrome.
2. I clicked on the **“Create Apps!”** button to begin my project.



3. Then, I clicked on **Projects** → **Start new project** and entered the name **“LED_Blink”**, followed by pressing **OK**.
4. I selected **Label** from the **User Interface** menu and dragged it onto the mobile screen layout.
 - A. I set **Width** to *Fill Parent*.
 - B. I set **TextAlignment** to *Center:1*.
 - C. I entered the text **“This App is made for Bluetooth-based control”**.

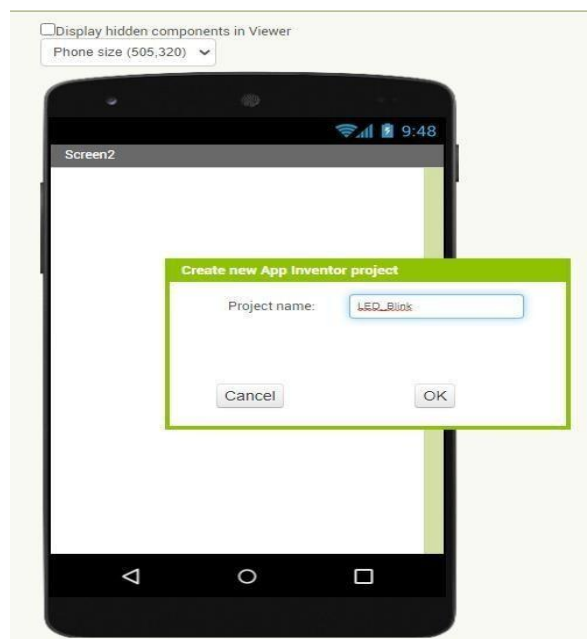
5. I added a **ListPicker** component from the **User Interface** menu and placed it on the screen.

A. I typed “**Connect to Device**” in the Text box.



6. I selected a **Button** component and dragged it onto the mobile screen.

A. I renamed its text to “**On**”.



7. I added another **Button** and renamed it to “**Off**”.
8. I placed another **Label** on the screen and typed “**Delete Everything**” in the text box.
9. I selected **Clock** from the **Sensor** menu and added it to the mobile screen.
10. I opened the **Connectivity** menu, selected **BluetoothClient**, and added it to the mobile screen.

11. Then, I clicked on the **Blocks** button to start programming:

- a. I created a block for **ListPicker1.AfterPicking**, where I connected the Bluetooth device using the **BluetoothClient1.Connect address** block and **ListPicker1.Selection**.
- b. I created another block for **ListPicker1.BeforePicking** to set device elements using **BluetoothClient1.AddressesAndNames**.
- c. I created a **Button1.Click** block to send text “1” through **BluetoothClient1.SendText** (for turning ON LED).
- d. I created a **Button2.Click** block to send text “0” (for turning OFF LED).
- e. Finally, I used the **Clock1.Timer** block with **if-then** logic to check if **BluetoothClient1** is connected and to receive text data using **BluetoothClient1.ReceiveText** and **BluetoothClient1.BytesAvailableToReceive**.

Result:

I successfully created a Bluetooth-controlled mobile application using MIT App Inventor and established a connection with Arduino. The app was able to send and receive data wirelessly between the smartphone and the microcontroller.



Conclusion:

From this experiment, I learned how to integrate Bluetooth communication with Arduino and Android applications. This setup allows wireless data transmission, which is essential in IoT-based projects for real-time monitoring and device control.

Experiment 7

Aim:

To interface Bluetooth with Arduino/Raspberry Pi and write a program to turn the LED ON or OFF when '1' or '0' is received from a smartphone using Bluetooth.

Description:

In this experiment, I established Bluetooth communication between a smartphone and the Arduino microcontroller to achieve wireless control of an LED. A mobile application was developed using MIT App Inventor to transmit control commands through Bluetooth. The Arduino was programmed to turn the LED ON when it received the value '1' and OFF when it received the value '0'. This experiment helped me understand the fundamentals of Bluetooth communication, mobile-based device control, and wireless connectivity in IoT applications.

Procedure:

- 1) I opened **MIT App Inventor** in Google Chrome.
- 2) I clicked on the **"Create Apps!"** button to begin developing my application.
- 3) I selected the **"Blocks"** button to start programming the logic for Bluetooth communication.
 - a)
 1. I clicked on **ListPicker1** and selected **"when ListPicker1.AfterPicking do"**.
 2. I clicked on **Control** and selected the **"if then"** option.
 3. I clicked on **BluetoothClient1** and selected **"call BluetoothClient1.Connect address"**.
 4. I clicked on **ListPicker1** again and chose **"ListPicker1.Selection"**.
 5. Then, I clicked on **ListPicker1** and selected **"set ListPicker1.Elements to"**.
 6. Finally, I clicked on **BluetoothClient1** and selected **"BluetoothClient1.AddressesAndNames"**.
 - b)
 1. I clicked on **ListPicker1** and selected **"when ListPicker1.BeforePicking do"**.
 2. I clicked on **ListPicker1** and chose **"set ListPicker1.Elements to"**.
 3. Then, I clicked on **BluetoothClient1** and selected **"BluetoothClient1.AddressesAndNames"**.

c)

1. I clicked on **Button1** and selected “**when Button1.Click do**”.
2. I clicked on **BluetoothClient1** and selected “**call BluetoothClient1.SendText text**”.
3. I clicked on **Text** and entered “**1**” into the text box (to send the LED ON command).

d)

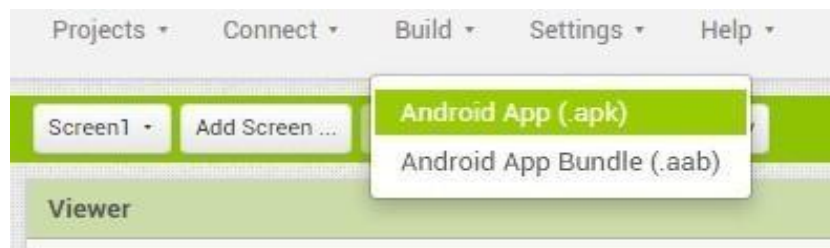
1. I repeated the above step for **Button2**, changing the text to “**0**” (to send the LED OFF command).

e)

1. I clicked on **Clock1** and selected “**when Clock1.Timer do**”.
2. I clicked on **Control** to add an “**if then**” block.
3. I clicked on **BluetoothClient1** and selected “**BluetoothClient1.IsConnected**”.
4. I clicked on **Label2** and selected “**set Label2.Text to**”.
5. I then clicked on **BluetoothClient1** and selected “**call BluetoothClient1.ReceiveText numberOfBytes**” and “**call BluetoothClient1.BytesAvailableToReceive**”.



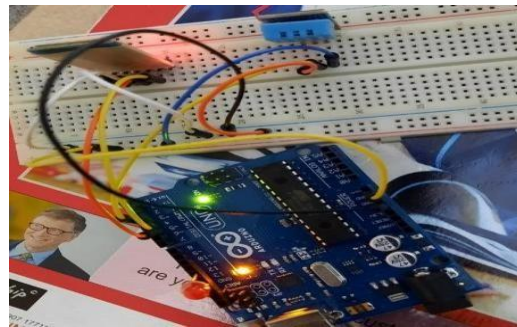
- 4) After completing the blocks, I clicked on **Build** and selected “**Android App (.apk)**” from the dropdown menu.
- 5) Once the build was completed, a **QR Code** was generated.





- 6) I scanned the QR Code on my smartphone and installed the “LED_Blink” application.
- 7) Finally, I connected the smartphone to the Arduino via Bluetooth and successfully controlled the LED — turning it ON when ‘1’ was sent and OFF when ‘0’ was sent.

Result:



I successfully interfaced the Bluetooth module with Arduino and controlled the LED through my smartphone using the custom MIT App Inventor mobile app. The LED turned ON when I sent ‘1’ and turned OFF when I sent ‘0’

Conclusion:

From this experiment, I learned how to establish wireless communication between a smartphone and Arduino using Bluetooth. This experiment helped me understand real-time control of hardware devices using mobile applications, which is an essential concept in IoT-based systems.

Experiment 8

Aim:

To write a program on Arduino/Raspberry Pi to upload temperature and humidity data to the ThingSpeak cloud.

Description:

In this experiment, I interfaced a DHT11 sensor with the Arduino to measure temperature and humidity. The sensor readings were collected by the Arduino and uploaded to the ThingSpeak cloud platform using the ESP8266 Wi-Fi module. ThingSpeak enabled the real-time visualization and monitoring of the collected data. Through this experiment, I gained a better understanding of integrating sensors, microcontrollers, wireless communication, and cloud platforms for IoT-based data monitoring applications.

Procedure:

1. I opened the **Arduino IDE** on my computer and started typing the following program.

Code:

```
#include <DHT.h>
#include <DHT_U.h>
#include <ThingSpeak.h>
#include <ESP8266WiFi.h>

const char *ssid = "xxxxxxx"; // my network SSID (WiFi name)
const char *pass = "xxxxxxx"; // my network password
unsigned long chid = 1648756; // my ThingSpeak channel ID
const char *key = "WWB8GVQ941FIXU8W"; // my Write API key

DHT Sensor(5, DHT11);
WiFiClient espclient;

void setup() {
  Sensor.begin();
  ThingSpeak.begin(espclient);
  Serial.begin(9600);
  Serial.println("Connecting with " + String(ssid));
  WiFi.begin(ssid, pass);
  while (WiFi.status() != WL_CONNECTED) {
    Serial.println(" ... ");
    delay(150);
  }
  Serial.println("\nWiFi Connected");
  Serial.println(WiFi.localIP());
}

void loop() {
  float tempr = Sensor.readTemperature();
```

```

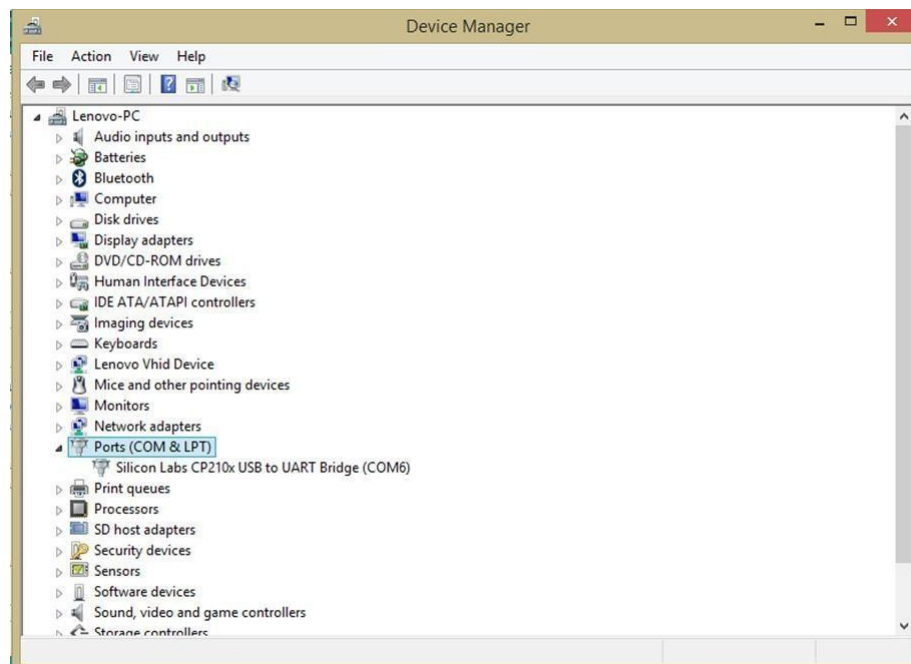
float humid = Sensor.readHumidity();

ThingSpeak.setField(1, tempr);
ThingSpeak.setField(2, humid);

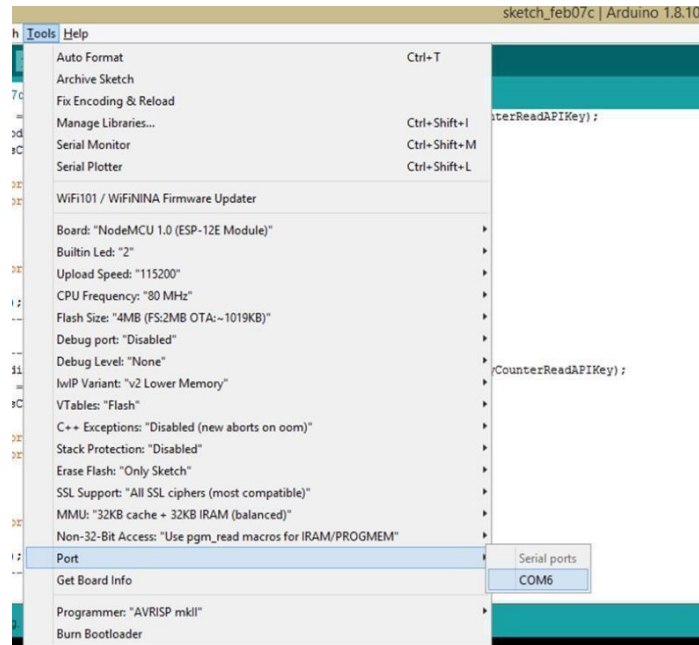
int x = ThingSpeak.writeFields(chid, key);
if (x == 200) {
    Serial.println("Channel update successful.");
} else {
    Serial.println("Problem updating channel. HTTP error code " + String(x));
}
delay(15000);
}

```

2. I saved the program with a suitable name in the Arduino IDE.
3. Then, I opened the **ThingSpeak website** (<https://thingspeak.com>) and created a new account.
 - a. I logged into my ThingSpeak account and clicked on “**New Channel.**”
 - b. In the **Channel Name** field, I entered the experiment name.
 - c. I entered “**Temperature**” in *Field 1* and “**Humidity**” in *Field 2*.
 - d. I clicked on the “**Save Channel**” button to create my data channel.
4. I reopened my Arduino program and clicked on the **Compile** button to verify it.
5. I connected the **Arduino board** to my computer using a USB cable.
6. I opened the **Device Manager** on my computer to check the COM port assigned to my Arduino board.
7. In the **Arduino IDE**, I selected **Tools** → **Port** and chose the available port (COM3, COM4, COM5, or COM6).

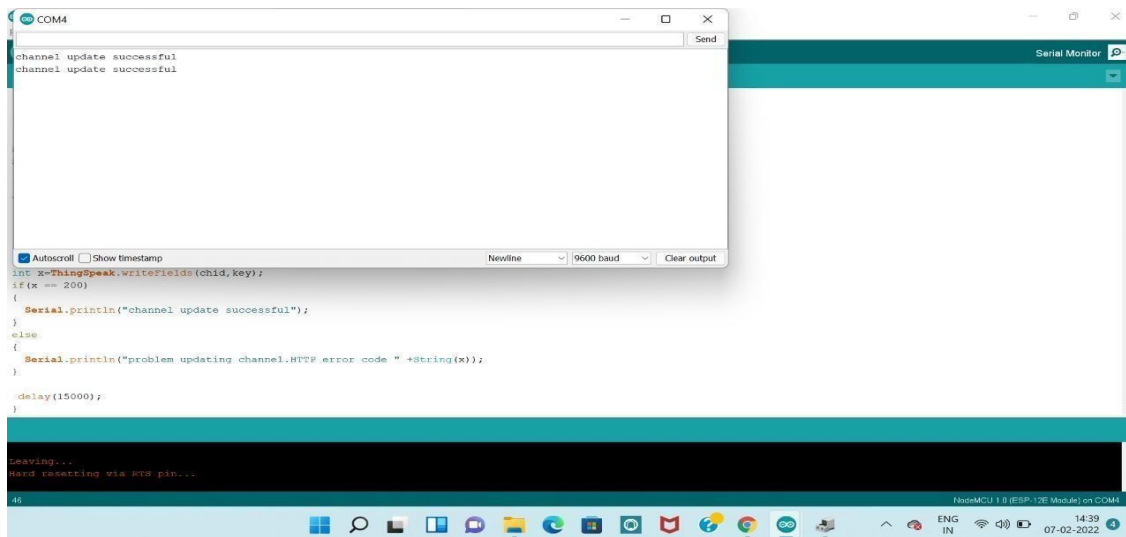


8. I then clicked the **Upload** button to upload the code to my Arduino board.

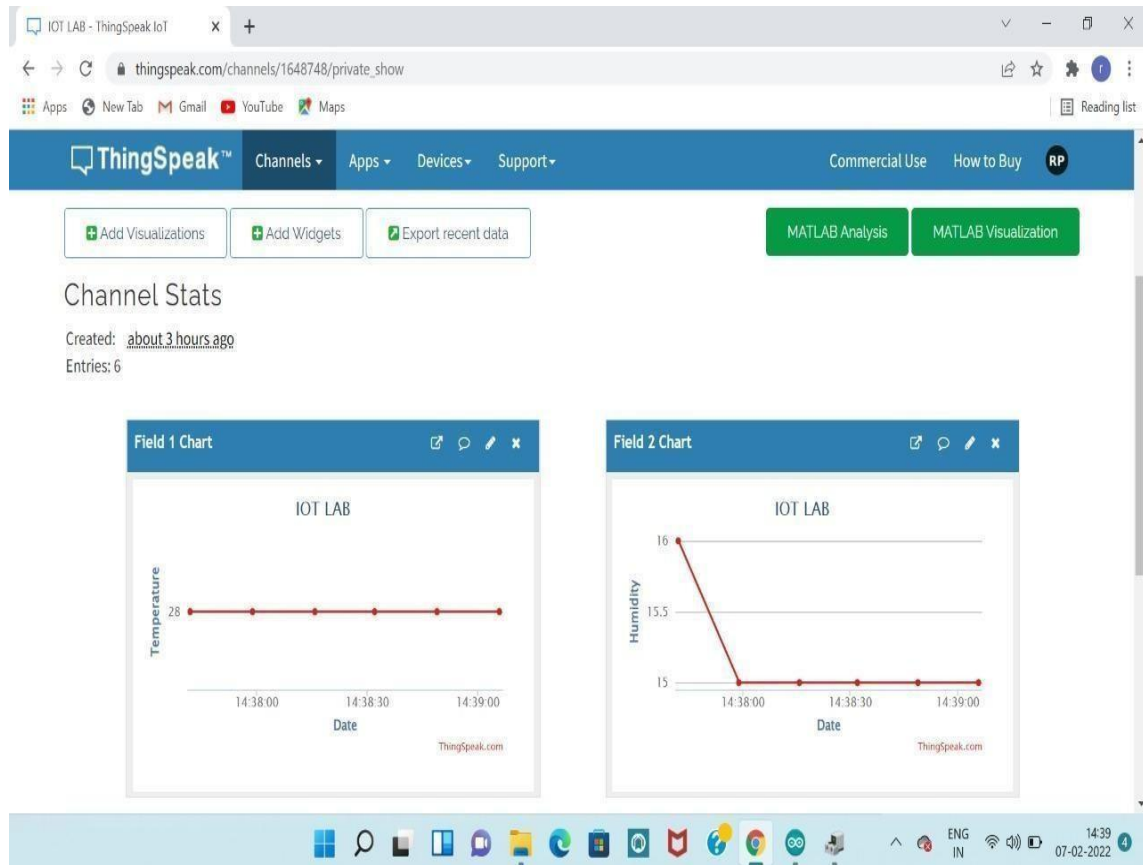


9. After uploading, I opened the **Serial Monitor** to check the connection status and confirm that the data was being sent to ThingSpeak.
10. I then opened my ThingSpeak account, navigated to the **Channels** section, and clicked on **Private View** to visualize the uploaded temperature and humidity data graphs in real-time.

Result:



I successfully uploaded real-time temperature and humidity readings from the DHT11 sensor to the ThingSpeak cloud platform. The data was displayed on the ThingSpeak channel as graphical plots for both temperature and humidity.



Conclusion:

Through this experiment, I learned how to connect an IoT device to a cloud platform for real-time data monitoring. I understood how to configure ThingSpeak channels, send sensor data using the ESP8266 WiFi module, and visualize environmental parameters online. This experiment demonstrated a complete IoT workflow — from sensing and connectivity to cloud-based data visualization.

Experiment 9

Aim:

To write a program on Arduino/Raspberry Pi to retrieve temperature and humidity data from the ThingSpeak cloud.

Description:

In this experiment, I connected an Arduino with Wi-Fi capability to the ThingSpeak cloud platform to retrieve stored temperature and humidity data. Using the APIs provided by ThingSpeak, I accessed the required data fields from a specific channel and successfully obtained the sensor readings. This experiment helped me understand how IoT devices communicate with cloud platforms to both retrieve and process data, enabling further analysis, monitoring, and automation.

Procedure:

1) I opened the **Arduino IDE** on my computer and typed the following program:

Code:

```
#include "ThingSpeak.h"
#include <ESP8266WiFi.h>

const char ssid[] = "xxxxxxx"; // my network SSID (WiFi name)
const char pass[] = "xxxxxxx"; // my network password

WiFiClient client;

// ----- Channel Details----- //
unsigned long counterChannelNumber = 1648756; // Channel ID
const char *myCounterReadAPIKey = "ODGH1JHLPGDUG39W"; // Read API Key
const int FieldNumber1 = 1; // Field for Temperature
const int FieldNumber2 = 2; // Field for Humidity
// ----- //

void setup() {
    Serial.begin(9600);
    WiFi.mode(WIFI_STA);
    ThingSpeak.begin(client);
}

void loop() {
    // ----- Network ----- //
    if (WiFi.status() != WL_CONNECTED) {
        Serial.print("Connecting to ");
        Serial.print(ssid);
        Serial.println(" .... ");
        while (WiFi.status() != WL_CONNECTED) {
            WiFi.begin(ssid, pass);
            delay(5000);
        }
    }
}
```

```

    }
    Serial.println("Connected to Wi-Fi successfully.");
}
// ----- End of Network connection ----- //

// ----- Channel 1 (Temperature) ----- //
long temp = ThingSpeak.readLongField(counterChannelNumber, FieldNumber1,
myCounterReadAPIKey);
int statusCode = ThingSpeak.getLastReadStatus();
if (statusCode == 200) {
    Serial.print("Temperature: ");
    Serial.println(temp);
} else {
    Serial.println("Unable to read channel / No internet connection");
}
delay(100);

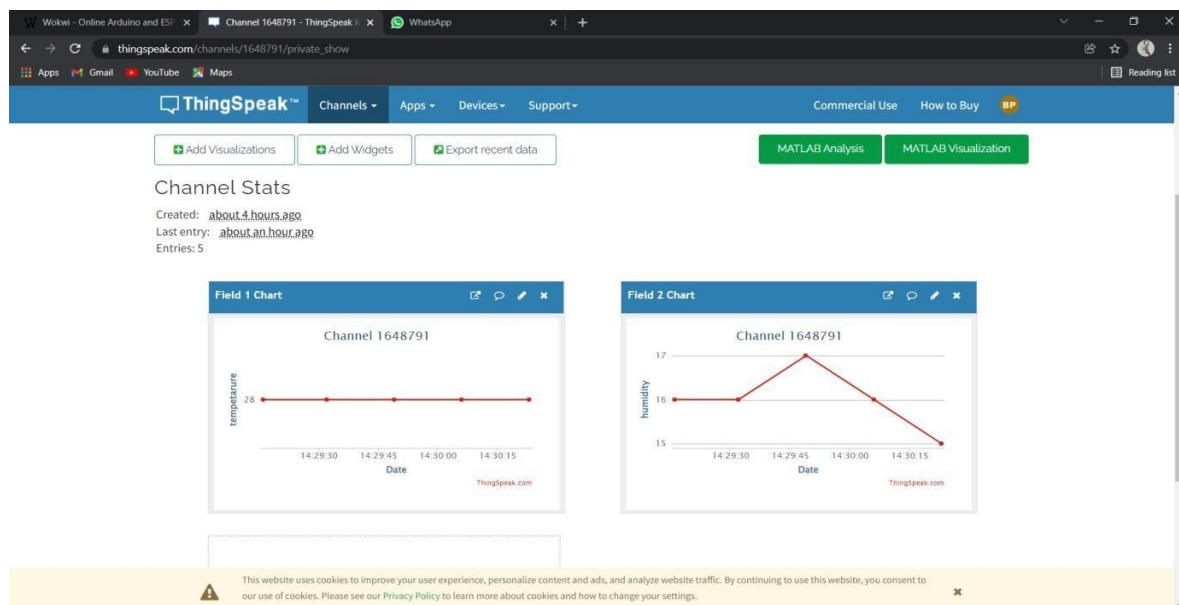
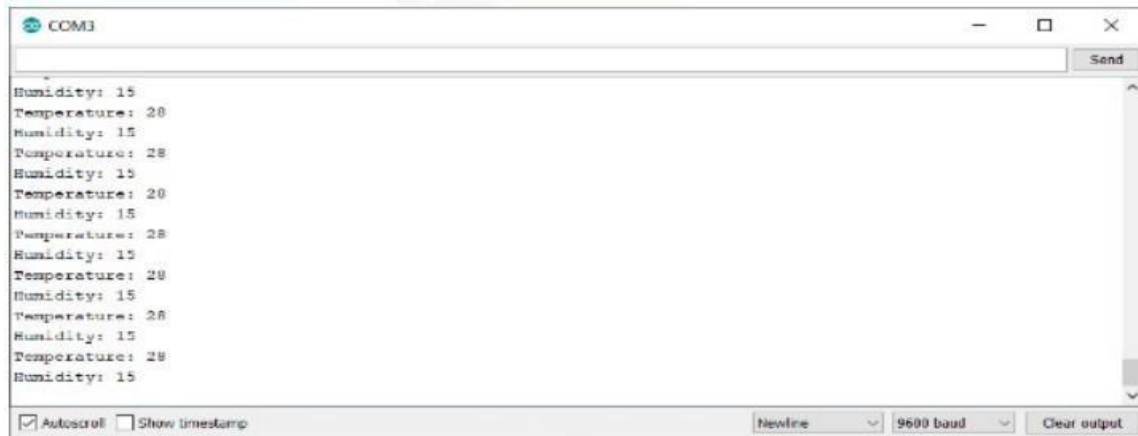
// ----- Channel 2 (Humidity)----- //
long humidity = ThingSpeak.readLongField(counterChannelNumber, FieldNumber2,
myCounterReadAPIKey);
statusCode = ThingSpeak.getLastReadStatus();
if (statusCode == 200) {
    Serial.print("Humidity: ");
    Serial.println(humidity);
} else {
    Serial.println("Unable to read channel / No internet connection");
}
delay(100);
}

```

- 2) I saved the program in my Arduino IDE.
- 3) I opened the **ThingSpeak website** (<https://thingspeak.com>) and created a new account.
 - I logged into my ThingSpeak account and clicked on “**New Channel.**”
 - In the **Channel Name** field, I entered the experiment name.
 - I entered “**Temperature**” in *Field 1* and “**Humidity**” in *Field 2*.
 - I clicked on “**Save Channel**” to create the channel.
- 4) I reopened my Arduino program and clicked on the **Compile** button to verify it.
- 5) I connected the **Arduino board** to my computer using a **USB cable**.
- 6) I opened the **Device Manager** on my system to check the COM port assigned to the board.
- 7) I selected the correct port under **Tools** → **Port** in the Arduino IDE (COM3, COM4, COM5, or COM6).
- 8) I then clicked on the **Upload** button to upload the program to my Arduino board.
- 9) Once uploaded, I opened the **Serial Monitor** to observe the retrieved data from the ThingSpeak cloud.
- 10) Finally, I went to my ThingSpeak account, clicked on **Channels** → **Private View**, and confirmed that the data retrieved from the cloud matched the temperature and humidity values displayed in the Serial Monitor.

Result:

I successfully retrieved temperature and humidity readings stored in the ThingSpeak cloud and displayed them in the Arduino Serial Monitor. The system fetched the data in real-time using the Thing Speak Read API.



Conclusion:

From this experiment, I learned how to connect an Arduino to the internet using WiFi and fetch stored IoT sensor data from a cloud platform. This helped me understand the process of data retrieval using APIs, an essential part of cloud-based IoT systems. I also realized how ThingSpeak allows two-way communication — sending and receiving sensor data — between devices and the cloud.

Experiment 10

Aim:

To install MySQL database on Raspberry Pi and perform basic SQL queries.

Procedure:

Step 1: Update the Raspberry Pi Packages

Open the Raspberry Pi terminal and update the existing packages:

```
sudo apt-get update
```

Step 2: Install MySQL Server

Install the MySQL (or MariaDB) server using:

```
sudo apt-get install mysql-server
```

If prompted “Do you want to continue?”, type **y** and press **Enter**.

```
pi@raspberrypi: ~  
File Edit Tabs Help  
pi@raspberrypi:~ $ sudo apt-get install mysql-server  
Reading package lists... Done  
Building dependency tree  
Reading state information... Done  
The following additional packages will be installed:
```

Step 3: Secure the MySQL Installation

Run the secure installation command:

```
sudo mysql_secure_installation
```

Press **Enter** for the current root password.

Type **y** to set a new password, then enter your desired password.

Type y to:

- Remove anonymous users
- Disallow remote login

- Remove test databases
- Reload privilege tables

```

pi@raspberrypi:~$ sudo mysql_secure_installation

NOTE: RUNNING ALL PARTS OF THIS SCRIPT IS RECOMMENDED FOR ALL MariaDB
SERVERS IN PRODUCTION USE! PLEASE READ EACH STEP CAREFULLY!

In order to log into MariaDB to secure it, we'll need the current
password for the root user. If you've just installed MariaDB, and
you haven't set the root password yet, the password will be blank,
so you should just press enter here.

Enter current password for root (enter for none):
  
```

Step 4: Log in to MySQL Server

`sudo mysql -u root -p`

Enter the password you set during the secure installation.

```

pi@raspberrypi:~$ sudo mysql_secure_installation

NOTE: RUNNING ALL PARTS OF THIS SCRIPT IS RECOMMENDED FOR ALL MariaDB
SERVERS IN PRODUCTION USE! PLEASE READ EACH STEP CAREFULLY!

In order to log into MariaDB to secure it, we'll need the current
password for the root user. If you've just installed MariaDB, and
you haven't set the root password yet, the password will be blank,
so you should just press enter here.

Enter current password for root (enter for none): OK, successfully used password, moving on...
Setting the root password ensures that nobody can log into the MariaDB
root user without the proper authorization.

Set root password? [Y/n] y
  
```

```

Set root password? [Y/n] y
New password:
Re-enter new password:
Password updated successfully!
Reloading privilege tables..
... Success!

By default, a MariaDB installation has an anonymous user, allowing anyone
to log into MariaDB without having to have a user account created for
them. This is intended only for testing, and to make the installation
go a bit smoother. You should remove them before moving into a
production environment.

Remove anonymous users? [Y/n]
  
```

Step 5: Show Available Databases

`show databases;`

```

MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
+-----+
3 rows in set (0.01 sec)

MariaDB [(none)]>
  
```

Step 6: Create a New Database

```
MariaDB [(none)]> CREATE DATABASE Demo;  
Query OK, 1 row affected (0.00 sec)
```

A screenshot of a MySQL terminal window. The prompt is 'MariaDB [(none)]>'. The user has entered 'CREATE DATABASE Demo;'. The output is 'Query OK, 1 row affected (0.00 sec)'. A white arrow points from the text 'database name' to the word 'Demo' in the command.

CREATE DATABASE Demo;

Step 7: Select the Database

USE Demo;

```
MariaDB [(none)]> USE Demo;  
Database changed  
MariaDB [Demo]> |
```

A screenshot of a MySQL terminal window. The prompt is 'MariaDB [(none)]>'. The user has entered 'USE Demo;'. The output is 'Database changed'. The prompt has changed to 'MariaDB [Demo]>'. There is a vertical bar at the end of the line.

Step 8: Create a Database User

CREATE USER 'admin'@'localhost' IDENTIFIED BY 'admin';

```
MariaDB [(none)]> USE Demo;  
Database changed  
MariaDB [Demo]> CREATE USER 'admin'@'localhost' IDENTIFIED BY 'admin';  
Query OK, 0 rows affected (0.01 sec)
```

A screenshot of a MySQL terminal window. The prompt is 'MariaDB [(none)]>'. The user has entered 'USE Demo;'. The output is 'Database changed'. The prompt has changed to 'MariaDB [Demo]>'. The user has entered 'CREATE USER 'admin'@'localhost' IDENTIFIED BY 'admin';'. The output is 'Query OK, 0 rows affected (0.01 sec)'. Two white arrows point from labels 'User name' and 'password' to the 'admin' parts of the command.

Step 9: Grant Privileges

GRANT ALL PRIVILEGES ON Demo.* TO 'admin'@'localhost';

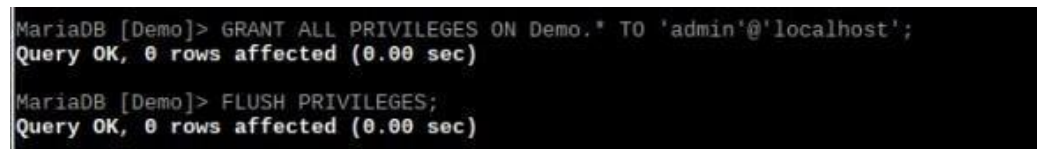
```
MariaDB [(none)]> USE Demo;  
Database changed  
MariaDB [Demo]> CREATE USER 'admin'@'localhost' IDENTIFIED BY 'admin';  
Query OK, 0 rows affected (0.01 sec)  
  
MariaDB [Demo]> GRANT ALL PRIVILEGES ON Demo.* TO 'admin'@'localhost';  
Query OK, 0 rows affected (0.00 sec)
```

A screenshot of a MySQL terminal window. The prompt is 'MariaDB [(none)]>'. The user has entered 'USE Demo;'. The output is 'Database changed'. The prompt has changed to 'MariaDB [Demo]>'. The user has entered 'CREATE USER 'admin'@'localhost' IDENTIFIED BY 'admin;'. The output is 'Query OK, 0 rows affected (0.01 sec)'. The prompt is 'MariaDB [Demo]>'. The user has entered 'GRANT ALL PRIVILEGES ON Demo.* TO 'admin'@'localhost;'. The output is 'Query OK, 0 rows affected (0.00 sec)'. There is a blank line before the final command.

Step 10: Save Changes

FLUSH PRIVILEGES;

```
MariaDB [Demo]> GRANT ALL PRIVILEGES ON Demo.* TO 'admin'@'localhost';  
Query OK, 0 rows affected (0.00 sec)  
  
MariaDB [Demo]> FLUSH PRIVILEGES;  
Query OK, 0 rows affected (0.00 sec)
```

A screenshot of a MySQL terminal window. The prompt is 'MariaDB [Demo]>'. The user has entered 'GRANT ALL PRIVILEGES ON Demo.* TO 'admin'@'localhost;'. The output is 'Query OK, 0 rows affected (0.00 sec)'. There is a blank line. The prompt is 'MariaDB [Demo]>'. The user has entered 'FLUSH PRIVILEGES;'. The output is 'Query OK, 0 rows affected (0.00 sec)'. There is a blank line.

Step 11: Exit MySQL

quit;

```
MariaDB [Demo]> quit  
Bye  
pi@raspberrypi:~ $
```

Step 12: Restart MySQL Server

sudo service mysql restart

```
pi@raspberrypi:~ $ sudo service mysql restart  
pi@raspberrypi:~ $
```

Performing Basic SQL Queries

1. Log in to MySQL Again

sudo mysql -u root -p

```
pi@raspberrypi:~ $ sudo mysql -u root -p  
Enter password:  
Welcome to the MariaDB monitor.  Commands end with ; or \g.  
Your MariaDB connection id is 2  
Server version: 10.1.38-MariaDB-0+deb9u1 Raspbian 9.0  
  
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
  
MariaDB [(none)]> use Demo  
Database changed  
MariaDB [Demo]> create table login(  
  -> username varchar(25),password varchar(25));  
Query OK, 0 rows affected (0.05 sec)  
  
MariaDB [Demo]>
```

2. Use the Demo Database

USE Demo;

```
MariaDB [Demo]> insert into login values ('admin','admin123');  
Query OK, 1 row affected (0.00 sec)  
  
MariaDB [Demo]>
```

Output:

MariaDB [(none)]> use Demo;Database changed

3. Create a Table

Example 1 — Login table:

```
CREATE TABLE login(username VARCHAR(25), password VARCHAR(25));
```

Example 2 — Data table:

```
CREATE TABLE data(sno INT(11), temp VARCHAR(25), hum VARCHAR(25));
```

Output:

Query OK, 0 rows affected (0.06 sec)

4. Insert Data

```
INSERT INTO login VALUES('admin', 'admin123');INSERT INTO login  
VALUES('user', 'user123');
```

Output:

Query OK, 1 row affected (0.01 sec)

5. View the Data

```
SELECT * FROM login;
```

6. Update Data

```
UPDATE data SET temp='26.4' WHERE sno=1;
```



```
MariaDB [Demo]> select * from login;  
+-----+  
| username | password |  
+-----+  
| admin    | admin123 |  
+-----+  
1 row in set (0.01 sec)  
MariaDB [Demo]>
```

Result:

MySQL database was successfully installed on Raspberry Pi, and basic SQL operations such as creating databases, inserting, updating, and deleting records were performed successfully.

Experiment 11

Aim:

To write a program on Arduino/Raspberry Pi to publish temperature data to an MQTT broker.

Procedure:

Step 1: Install Raspberry Pi OS

Download and install Raspberry Pi OS from:

<https://www.raspberrypi.com/software/>

Step 2: Open the Raspberry Pi Terminal and Write the Program

Program:

```
import sysimport Adafruit_DHTimport timefrom paho.mqtt.client import *  
# Create MQTT client  
client = Client("Device1")  
# Connect to MQTT broker  
c = client.connect('broker.hivemq.com', 1883)  
if c == 0:  
    print('Client connected')  
# Read and publish temperature and humidity data continuouslywhile True:  
    hum, temp = Adafruit_DHT.read_retry(11, 4)  
    print('Temperature:', temp)  
    print('Humidity:', hum)  
  
    message = 'temp=' + str(temp) + ",hum=" + str(hum)  
    client.publish('iotdata', message)  
  
    time.sleep(2)
```

Step 3: Save the Program

Save the file with the name **dhtpublish.py**

Output:

Open terminal and run:

```
pi@raspberrypi:~ $ cd iot  
pi@raspberrypi:~/iot $ python3 dhtpublish.py
```

client connectedTemperature: 28.0Humidity: 95.0Temperature: 29.0Humidity: 94.0Temperature: 28.0Humidity: 94.0Temperature: 29.0Humidity: 95.0Temperature: 28.0Humidity: 94.0Temperature: 29.0Humidity: 94.0Temperature: 28.0Humidity: 93.0

To View Results on Smartphone:

- Download **MQTT Dash** app from the Play Store.
- Open the app and tap on the “+” icon.

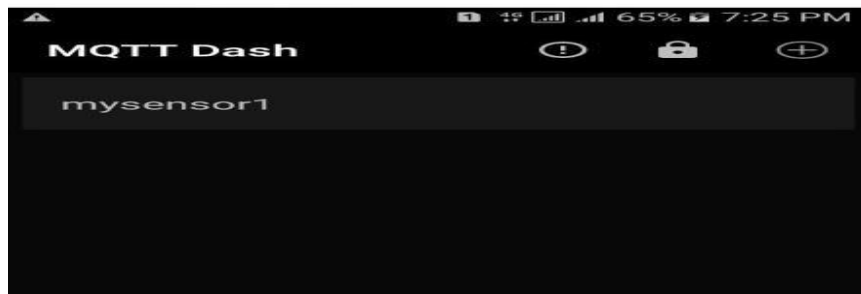
Enter:

Name: mysensor1

Address: broker.hivemq.com

Save the connection.

Tap on **mysensor1** to view live temperature and humidity updates.

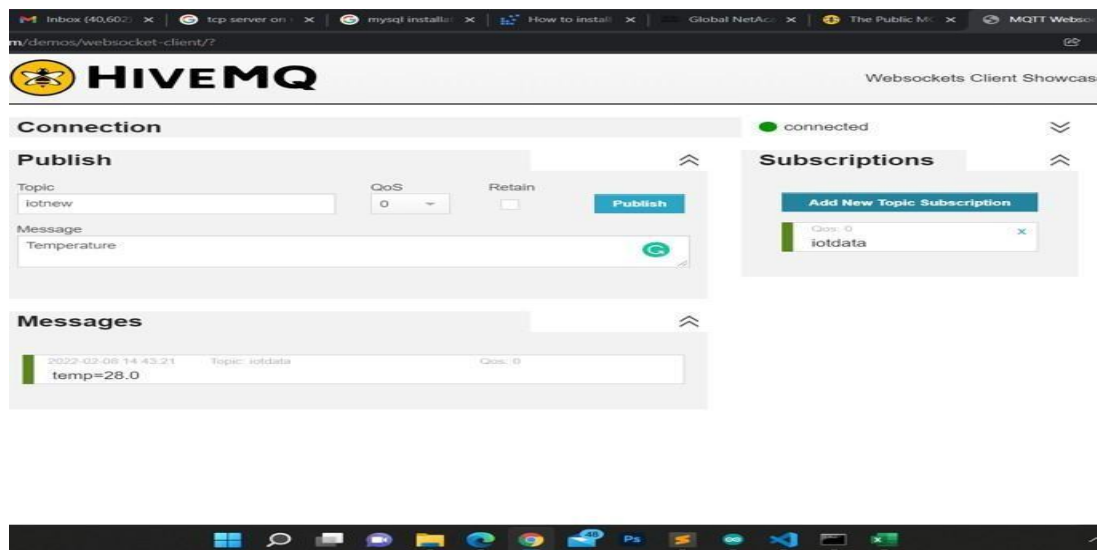
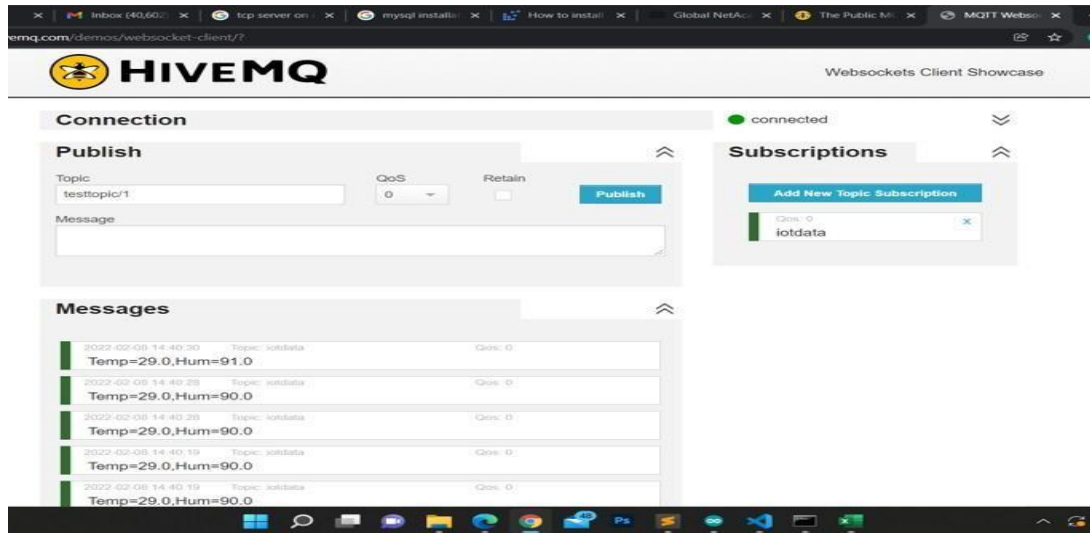


To View Results on Website:

Open the following link in **Google Chrome**:

<https://www.hivemq.com/demos/websocket-client/>

You will be able to see the published data from your Raspberry Pi under the topic “**iotdata**”.



Result:

The program successfully reads temperature and humidity data using a DHT sensor and publishes it to the **MQTT broker (broker.hivemq.com)**. The data can be monitored both through the **MQTT Dash mobile app** and the **HiveMQ web client** in real time.

Experiment 12

Aim:

To write a program on Arduino/Raspberry Pi to subscribe to an MQTT broker for temperature data and print it.

Procedure:

Step 1:

I downloaded and installed the Raspberry Pi OS from the official website:
<https://www.raspberrypi.com/software/>

Step 2:

I opened the Raspberry Pi terminal and wrote the following program.

Program:

```
import sys
import Adafruit_DHT
import time
from paho.mqtt.client import *

# Function to handle incoming messages
def on_message(client, userdata, msg):
    print('Channel:', msg.topic)
    message = str(msg.payload.decode('utf-8'))
    print('Message:', message)

    hum, temp = Adafruit_DHT.read_retry(11, 4)
    t = 'temp=' + str(temp)
    h = 'hum=' + str(hum)

    if message == 'Temperature':
        client.publish('iotdata', t)
    if message == 'Humidity':
        client.publish('iotdata', h)

# MQTT client setup
client = Client("Device1")
c = client.connect('broker.hivemq.com', 1883)
if c == 0:
    print('Client connected')

client.subscribe('iot')
client.on_message = on_message
client.loop_forever()
```

Step 3:

I saved the program with the name **dht.py** and executed it in the terminal.

Command:

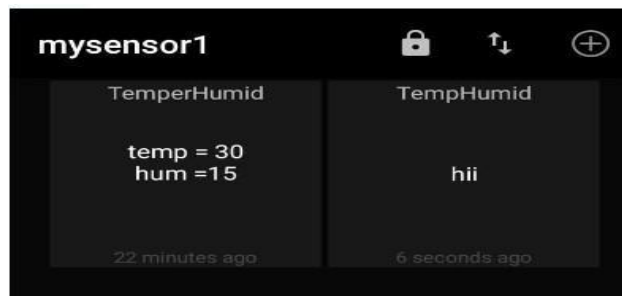
```
python3 dht.py
```

Output:

```
client connectedChannel: iotnewMessage: Temperature
28.0Channel: iotnewMessage: Humidity
```

To View the Result on Smartphone:

1. I downloaded the **MQTT Dash** app from the Play Store.
2. I opened the app and clicked on the “+” icon.
3. I entered:
 - **Name:** mysensor1
 - **Address:** broker.hivemq.com
4. I saved the connection.
5. Finally, I clicked on **mysensor1** to view live temperature and humidity updates sent from the Raspberry Pi.



Result:

I successfully wrote and executed a Python program on Raspberry Pi to subscribe to an **MQTT broker (broker.hivemq.com)** for temperature and humidity data. The subscribed messages were displayed on the terminal and could also be viewed on the **MQTT Dash mobile app**.